锁存器 寄存器与计数器实验

依次完成①D 锁存器与 D 触发器对比、②两级寄存器、③模 10 计数器，理解状态存储、非阻塞赋值及两种复位。三题提交一份报告。橙色框为必做任务，黄色框为注意事项。

开始实验

第一题运行提供的完整示例，观察并分析；第二、三题补全 lab_rtl.v 中对应的 TODO。保留模块名、端口和敏感列表；testbench 原样使用，无需理解其内部实现。

创建一个仿真工程：rtl 中两份 .v 文件加入 Design Sources，sim 中四份 .v 文件加入 Simulation Sources。必做使用 tb_storage、tb_pipeline、tb_counter；选做异步计数器使用 tb_counter_async。第一题代码已提供，只补全第二、三题对应的 TODO。

分别将下表 testbench 设为仿真顶层，运行 Run Behavioral Simulation → Run All。修改 RTL 或切换题目后，关闭旧仿真并重新启动。

题目	仿真顶层	Tcl Console 期望输出
1 锁存器与触发器	tb_storage	STORAGE PASS
2 两级寄存器	tb_pipeline	PIPELINE PASS
3 计数器	tb_counter *tb_counter_async	COUNTER_SYNC PASS *COUNTER_ASYNC PASS

注意事项 仿真判定

结论由 ========== 分隔，failed_checks=0 表示检查通过。tb_counter 仅检查必做的 counter_sync；tb_counter_async 仅检查选做的 counter_async，两个测试互不依赖。编译报错、TIMEOUT 或没有最终结论均不能记为通过。

统一提交

自行编写一份 Word 实验报告，文件命名为 学号_姓名_时序逻辑实验四.docx，包含以下内容：

- 报告写明：姓名、学号、教室号、日期。
- 三题最终结论截图；每题一张支持结论的波形及 1 ~ 3 句分析。
- 第二、三题完成的 RTL，基础要求为 pipeline2 和 counter_sync；有余力可完成 counter_async。第一题提供的代码无需提交到报告中。提交代码时，文本保留缩进与高亮，或提供清晰截图。

提交截止时间：**10.19 周一 23:59**

必做题 1 D 锁存器与 D 触发器 20%

运行提供的 RTL 与 testbench，对比两种一位存储电路。无需补写代码；完成仿真截图与简要分析。

时序电路能够保存状态，输出不仅取决于当前输入，还与此前保存的状态有关。锁存器和触发器都能保存一位数据，区别在于更新时机。

两个模块均为 clk、d 输入和 q 输出，均为 1 位，无复位。dlatch 在 clk=1 期间**透明**、clk=0 时**保持**；dff 仅在 clk **上升沿采样** d，其余时间保持。

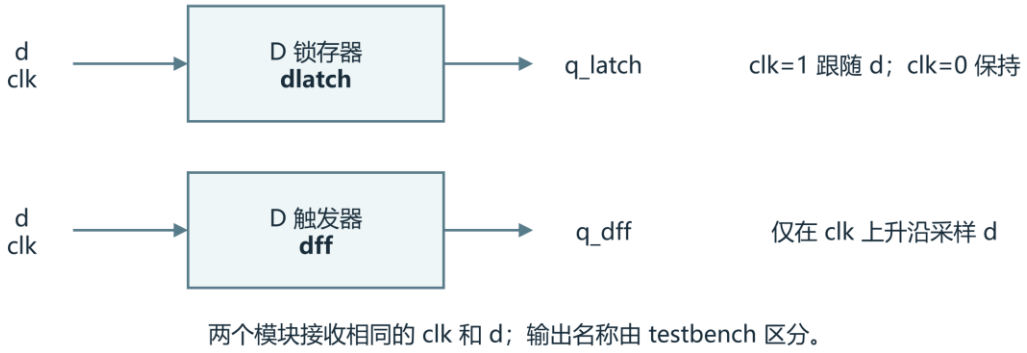
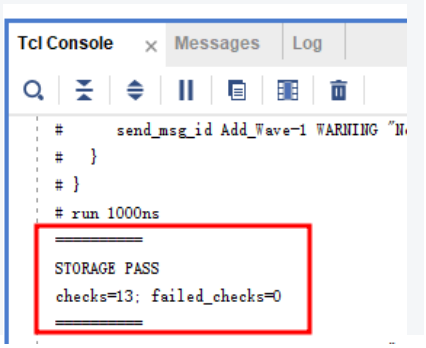


图 1 相同输入下的锁存器与触发器

提供的 RTL

```
`timescale 1ns/1ps
// D latch: transparent while clk is high, holds while clk is low.
module dlatch(input wire clk, input wire d, output reg q);
    always @(*) begin
        if (clk) q <= d;
    end
endmodule

// D flip-flop: samples d only on a rising clock edge.
module dff(input wire clk, input wire d, output reg q);
    always @(posedge clk) begin
        q <= d;
    end
endmodule
```



注意事项 电平敏感与边沿触发

锁存器使用 always @(*)，保持分支有意不赋值；触发器使用 always @(posedge clk)。均用 <=。首次采样前输出可能为 X。

观察与分析

以 tb_storage 为顶层运行 Run All，预期 STORAGE PASS、checks=13、failed_checks=0。完整测试源码见文末，保存为 sim/tb_storage.v。

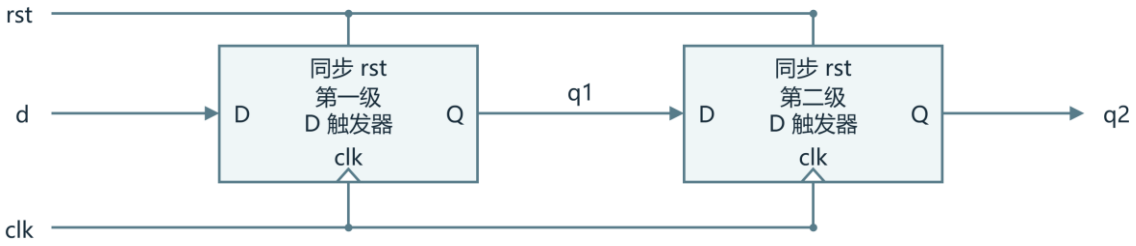
截取 5 ~ 70 ns 的 clk、d、q_latch、q_dff 波形。说明高电平期间 d 改变时两者的差异、低电平期间的保持，以及 40 ns 上升沿的采样。用具体时刻和数值支持结论，写 1 ~ 3 句即可。

必做题 2 两级寄存器与非阻塞赋值 40%

模块名：`pipeline2`。两个串联的 D 触发器，只在时钟上升沿更新，沿间保持。

设计规则：在 `always @(posedge clk)` 中，优先判断复位；`rst=1` 时将两个输出清零，否则让 `q1` 采样 `d`，`q2` 采样更新前的 `q1`。赋值使用 `<=`。

端口名	方向与位宽	含义
`clk`	输入，1 位	上升沿更新
`rst`	输入，1 位	高电平同步复位
`d`	输入，1 位	第一级输入
`q1`、`q2`	输出，各 1 位，`reg`	第一级、第二级保存的值



共用 clk 和高电平同步 rst；三角符号表示上升沿触发。
同一上升沿：q1 采样 d，q2 采样更新前的 q1。

图 2 两级寄存器的数据传递框图

为什么使用非阻塞赋值

```
q1 <= d;
q2 <= q1;
```

非阻塞赋值先计算右侧，再在当前仿真时刻的后续阶段更新左侧。因此第二句使用旧 `q1`，不会直接得到本拍的 `d`。

注意事项 赋值方式

例如上升沿前 `d=1`、`q1=0`、`q2=0`，更新后为 `q1=1`、`q2=0`。若按该顺序改成 `q1 = d; q2 = q1;`，第二句会使用已修改的 `q1`，不符合两级传递要求。

状态转移与初始化

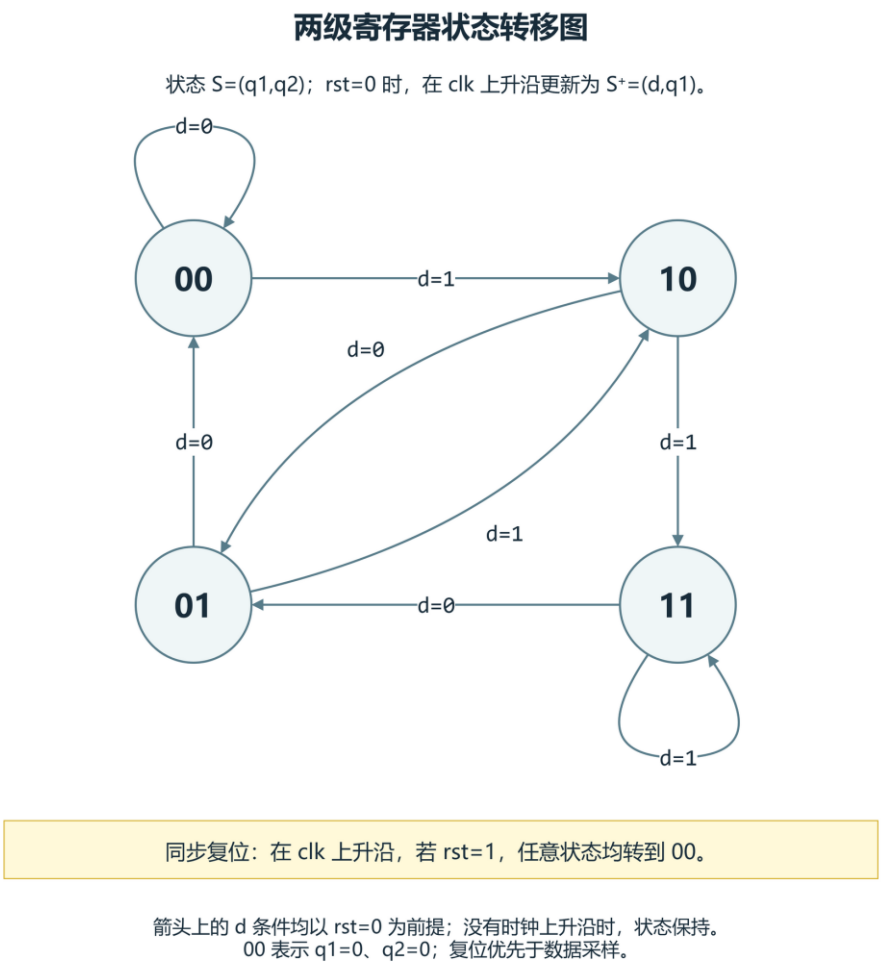


图 3 两级寄存器的状态转移图

图中列出输出已确定时的四种状态。同步复位须在 rst=1 时经过时钟上升沿，状态才变为 00。若未有效复位，初始状态可能为 XX；rst=0 且连续两拍 d=0 时，可依次得到 0X、00。

观察与分析

截取 20 ~ 60 ns 的 clk、rst、d、q1、q2 波形。选择 d 与旧 q1 不同的上升沿，说明更新后的两个输出，以及 q2 为什么没有直接得到本拍的 d；再指出一处沿间保持的现象。

本实验用到的语法

- module ... endmodule 定义模块；多个模块可并列写在同一文件中。reg 表示可在过程块中赋值，不保证综合为触发器，也可用于锁存器输出。
- `begin ... end` 包围多条语句；复位时要同时清零 q1、q2。
- `if (rst)` 判断复位；== 用于比较；<= 在这些过程块中用于非阻塞赋值。

必做题 3 模 10 计数器与复位 40%

完成 counter_sync（30%）；*选做 counter_async（10%）。固定 4 位模 10。

同步复位：rst=1 时，在 clk 上升沿清零。

*异步复位：rst 拉高时立即清零。

共同规则：未复位时上升沿加一，9 后回到 0；使用 4'd0、4'd1、4'd9 和 <=。

端口名	方向与位宽	含义
clk	输入，1 位	上升沿计数
rst	输入，1 位	高电平复位，优先于计数
count	输出，4 位，reg	计数范围 0 ~ 9

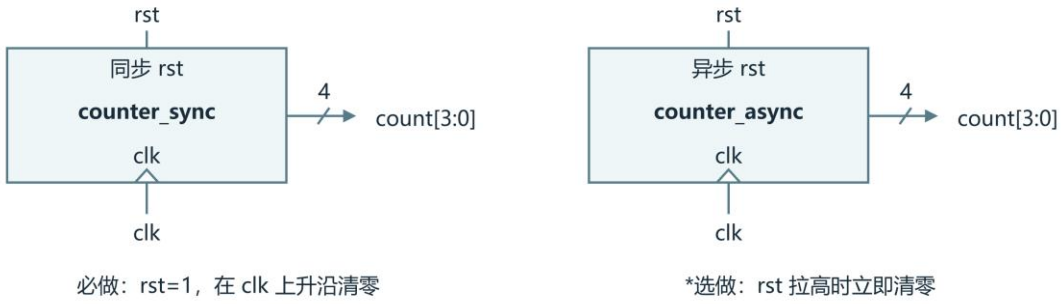


图 4 同步与异步复位计数器的模块框图

复位方式	敏感列表	复位何时生效
同步 counter_sync	@(posedge clk)	仅在时钟上升沿检查 rst
*异步 counter_async	@(posedge clk or posedge rst)	rst 拉高时即清零；时钟沿仍优先判断 rst

注意事项 复位响应

先完成同步版本，再复用计数规则完成异步版本。若复位脉冲完全处于两个上升沿之间，同步版本不会采到它。

有一些模块会出现低电平复位的情况，一般信号会进行说明，例如信号名为 rst_n 。

观察与分析

分析同步计数器的状态转移图（状态转移图参考第二题给出的例子），画图/表并添加合适的文字介绍。

截取 tb_counter 中 80 ~ 190 ns 的 clk、rst、count_sync（无符号十进制），说明 8→9→0 和同步复位。*选做时另截 tb_counter_async 中同一时段的 count_async，对照 140 ns 复位与 162 ~ 164 ns 短脉冲。

简单描述，你的模 10 计数器如何添加简单的外围电路变为模 7 计数器.

*选做：以 tb_counter_async 为顶层，获得 COUNTER_ASYNC PASS，并与同步版波形比较。必做使用 tb_counter，获得 COUNTER_SYNC PASS。两个测试各 checks=37、failed_checks=0。

第一题 testbench 源码

文件名 tb_storage.v。复制完整代码即可使用。

```
`timescale 1ns/1ps
module tb_storage;
    reg clk = 0, d = 0;
    wire q_latch, q_dff;
    integer checks = 0, errors = 0;
    dlatch u_latch(.clk(clk), .d(d), .q(q_latch));
    dff u_dff(.clk(clk), .d(d), .q(q_dff));

    task check;
        input expected_latch, expected_dff;
        begin
            #1; // Wait for nonblocking assignments to complete.
            checks = checks + 1;
            if ({q_latch, q_dff} !== {expected_latch, expected_dff}) begin
                errors = errors + 1;
                $display("FAIL t=%0t clk=%b d=%b latch=%b dff=%b",
                         $time, clk, d, q_latch, q_dff);
                $display("Expected: latch=%b dff=%b",
                         expected_latch, expected_dff);
            end
            #4;
        end
    endtask

    initial begin
        // Change d before clock edges, avoiding stimulus races.
        #5; clk = 1; check(0, 0); // 5 ns: first rising edge.
        d = 1; check(1, 0);       // 10 ns: high level, latch follows.
        d = 0; check(0, 0);       // 15 ns: high level, latch follows.
        d = 1; check(1, 0);       // 20 ns: dff still holds.
        clk = 0; check(1, 0);     // 25 ns: falling edge, both hold.
        d = 0; check(1, 0);       // 30 ns: low level, both hold.
        d = 1; check(1, 0);       // 35 ns: prepare before rising edge.
        clk = 1; check(1, 1);     // 40 ns: dff samples d=1.
        d = 0; check(0, 1);       // 45 ns: outputs differ again.
        clk = 0; check(0, 1);     // 50 ns: both hold.
        d = 1; check(0, 1);       // 55 ns: low level, both hold.
        d = 0; check(0, 1);       // 60 ns: prepare d=0.
        clk = 1; check(0, 0);     // 65 ns: dff samples d=0.
        $display("==========");
        if (errors == 0) $display("STORAGE PASS");
        else $display("STORAGE FAIL");
        $display("checks=%0d; failed_checks=%0d", checks, errors);
        $display("==========");
        $finish;
    end
    initial begin
        #1000;
        $display("==========\nSTORAGE FAIL\nReason: TIMEOUT\n==========");
        $finish;
    end
endmodule
```